

Ossim: OS-Driven Live Simulation for Cluster-Scale Full-Stack Evaluation

Yiliang Wan¹, Jonas Kaufmann², Yihan Yang¹, Chaoyi Ruan¹,
Haifeng Sun^{1*}, Antoine Kaufmann², Jialin Li¹

¹National University of Singapore

²Max Planck Institute for Software Systems

Abstract

Cluster-scale full-stack simulation is essential for evaluating distributed software stacks and emerging hardware components before deployment. Such simulation must achieve both full-stack fidelity for the unmodified production stack and the simulation performance required for iterative configuration exploration. However, no existing method achieves both. We present Ossim, an OS-level approach to cluster-scale full-stack simulation built on top of the Linux virtualization stack. Ossim comprises four subsystems: simulation-oriented scheduling, live memory hierarchy management, simulation-aware IPC, and multi-host coordination. Together, they coordinate live and modeled components under shared simulated time while controlling interference among co-located live VMs. These mechanisms point toward *simulation-native OS support*, where simulation control and orchestration become core OS responsibilities.

1 Introduction

Evaluating a distributed system requires understanding interactions across its stack: applications and runtimes, operating systems, processors and memory hierarchies, devices, and the network. These interactions determine whether a deployment configuration improves application performance and whether an emerging hardware capability delivers its expected benefits. Cluster-scale full-stack simulation enables such evaluation before hardware [1, 2, 9, 15, 25, 48, 49] is available at the target scale or production deployment is practical. It also facilitates choosing deployment configurations for production workloads such as cloud platforms [40, 51] and big-data frameworks [4, 5]. To support iterative exploration, however, a simulator must both preserve the behavior of the production stack and execute at practical speed.

Meeting these requirements presents two challenges. The first is hardware feature coverage. Production software can depend on specific processor instructions and features, architectural mechanisms, or device capabilities such as NIC hardware offloading. Faithfully evaluating such software requires the simulator to implement and precisely model the needed hardware features or functionalities. Existing simulators do not necessarily support the required features, even when they provide detailed models of other aspects of the

system. Disabling an unsupported feature or substituting a simplified implementation can change the workload.

The second challenge is the cost of detailed host simulation. Full-system simulators can execute unmodified operating systems, and modular frameworks such as SimBricks [32] and SplitSim [33] compose host, device, and network simulators into end-to-end systems. However, detailed CPU and memory-system simulation can dominate execution time and constrain the progress of the entire composition. Accurately modeling host performance requires capturing not only instruction execution but also cache misses, coherence traffic, memory contention, and their overlap with computation. Resolving these interactions in software is expensive. Simplifying them improves simulation speed but can distort application execution rates and, consequently, the load and timing observed by other components.

Live simulation offers a complementary approach that addresses both challenges. Live simulation executes available components directly on physical hardware. For features exposed through the live execution interface, it reuses the underlying hardware instead of requiring a simulator to implement that functionality. For processors and memory, native execution lets the physical hardware perform instruction execution, cache lookup, coherence, and memory access directly, avoiding the corresponding fine-grained simulation work. When the available hardware is representative of the host being studied, this preserves its native behavior while reserving explicit models for components or properties that differ from the target system.

However, native execution alone is not simulation. A VM’s progress depends on host scheduling and resource availability, while a modeled component advances according to its own timing model. Uncontrolled differences between these time bases can alter guest timers, communication timing, and distributed coordination. Likewise, using physical memory preserves the host’s memory behavior but does not automatically reproduce a different target latency. Combining live and modeled components therefore requires coordinated control over execution progress, resource performance, and the simulated times at which interactions become visible.

The key challenge is making virtualized hosts and heterogeneous modeled components composable under coordinated simulation control. Our focus is therefore a *composable substrate* in which VMs and modeled components share execution and communication abstractions, live resources

*Corresponding author.

can incorporate performance adjustments, and extending the mechanisms across physical hosts. These controls must work together: changing a vCPU’s effective execution rate must remain consistent with its guest-visible clock, its scheduling relative to other components, and its external interactions.

Our key insight is that the operating system with hardware virtualization support [41, 50, 53] is the natural integration point for these controls. The host OS encompasses both application and guest-kernel execution, allowing an unmodified software stack to participate in simulated time. Hardware virtualization retains native execution and access to exposed processor features, while the host controls vCPU scheduling, guest-visible clocks, and the resources backing guest memory. Virtual-device interfaces provide additional integration points for modeled communication and external components. Coordinating these mechanisms allows us to reuse native hardware behavior while controlling the progress and interactions of complete virtualized hosts.

We present Ossim, an OS-native substrate for full-stack live simulation at cluster scale. Ossim builds on the bases of prior live simulation systems [21, 30, 57], and follows two design principles. First, it integrates simulation control into OS mechanisms, preserving efficient native execution while enforcing simulated-time progress, resource-performance adjustments, and timed event delivery. Second, it defines common abstractions and interfaces that separate generic simulation control from component-specific implementations and performance models. This separation enables flexible composition across live and modeled components, host-backed resources, and communication models without redesigning the underlying control mechanisms. Our design comprises four major subsystems:

- *Simulation-oriented scheduling* represents live vCPUs and modeled components through a common virtual-time abstraction. Skew-bounded scheduling coordinates their progress, while gang scheduling supports live components that require coordinated execution.
- *Live memory hierarchy management* retains native memory accesses and coherence, manages their backing resources, and uses hardware-counter measurements to adjust virtual-time advancement for configured memory-latency differences.
- *Simulation-aware communication* provides common interfaces that connect live guests with built-in models or external simulators, and controls when messages become visible to receivers.
- *Multi-host coordination* extends progress tracking and timed communication across machines, allowing a simulation to use hardware resources beyond a single host.

We implement Ossim in the Linux virtualization stack and evaluate it with unmodified distributed services, databases, and analytics frameworks. Across seven reported end-to-end metrics, Ossim reproduces corresponding physical-system

measurements within 9%, while incurring only 2.6-24× wall-clock slowdown. In separate simulation-performance experiments, Ossim incurs 5.5-25× slowdown, compared with 665-1426× for a gem5 TimingSimple-based modular configuration and 3381-9329× for an O3-based configuration. These results demonstrate that coordinating native execution, live resources, and modeled interactions through OS support can make full-stack simulation practical without requiring detailed software simulation of the entire host.

2 Background and Motivation

2.1 Cluster-Scale Full-Stack Simulation

Cluster-scale full-stack simulation enables evaluating distributed software stacks and hardware configurations before the target system is available at scale or production deployment is practical. Such evaluation must capture both the functionality of individual components and their cross-layer interactions. For example, offloading network processing to a DPU changes not only packet-processing latency but also the CPU resources available to a distributed database. Evaluating its end-to-end benefit therefore requires capturing the production software stack together with processor, memory, device, and network behavior. A practical simulator must preserve these relevant behaviors while running fast enough to support iterative exploration. Therefore, we have two requirements for a cluster-scale full-stack simulator:

R1. Full-stack fidelity. We distinguish two complementary aspects of fidelity: full-stack coverage and accuracy.

- *Full-stack coverage* concerns the functionalities and interactions the simulation represents. On the software side, it requires executing the entire stack without source modification; on the hardware side, coverage requires the processor instructions, architectural mechanisms, and device capabilities exercised by that software. Replacing an end host with a traffic generator or disabling an unsupported hardware feature can change the software paths and cross-layer behavior being evaluated.
- *Accuracy* concerns how faithfully the represented system behaves relative to the target. Execution rates, resource contention, and communication timing must reflect the target closely enough to produce representative end-to-end results. For example, exposing an offload interface without capturing its effects on host CPU demand and packet-processing time would produce inaccurate results.

R2. Simulation performance. The simulator must complete end-to-end experiments fast enough for iterative configuration exploration. Users evaluate a candidate hardware, network, or software configuration, inspect its behavior, and refine it across repeated experiments. At cluster scale, each experiment executes a complete production stack on every simulated server, so the wall-clock time required for the overall simulation becomes critical: if a minute of target execution

requires days to simulate, exploring multiple configurations becomes impractical.

2.2 Related Simulation Methods

Discrete event simulation (DES). DES models the target system as a sequence of events processed in simulated time. Host simulators such as gem5 [18] and Simics [24] execute complete software stacks with models of processors, memory systems, and devices, while network simulators such as ns-3 [45] and OMNeT++ [55] model protocols and topologies. These tools provide detailed behavior within their supported configurations. However, a layer-specific simulator alone does not capture the interactions across an entire distributed system. Network simulations, for example, often use traffic generators or simplified hosts instead of executing the production software stack.

DES also presents two practical challenges. First, functional coverage depends on the simulator implementation, which may omit certain hardware instructions or mechanisms. Second, detailed timing models can be expensive to execute. For example, simulating one second of a 1,024-server fat-tree DCTCP workload in ns-3 can take over ten hours [6].

Modular composition. Modular frameworks [32, 33] connect independently developed component simulators through common interfaces and simulated-time synchronization. They enable host, device, and network models to form an end-to-end system while retaining their implementations.

Composition does not resolve limitations within those components. Required hardware functionality must still be supplied by a simulator, and the overall speed is determined by the slowest component simulator, which is typically end-host simulation. Modularity itself does not inherently remove the cost of detailed CPU and memory-system modeling.

Live simulation. Live simulation executes physically available components directly on hardware. This allows live simulation to reuse the hardware features, removing the dependency on software implementations. Native processor execution also lets hardware perform instruction execution, cache lookup, coherence, and memory access directly; this preserves the native behavior of processors without fine-grained host simulations. Explicit models are then only required for components that are unavailable.

However, live components execute according to physical scheduling and resource availability, whereas modeled components advance based on their timing models. Moreover, the available hardware may differ from the target in resource performance. To ensure accurate simulation, live simulation must control execution progress, account for resource differences, and coordinate when interactions become visible.

Prior systems establish key foundations for this approach. VM-based systems [21, 57] demonstrate execution of unmodified software stacks under controlled time, but with simulation control tailored only to particular settings. Process-

and container-based systems [29, 30, 43] provide virtual-time control for native applications, but leaves the guest OS and interactions with hardware outside the simulated stack. Other systems [36, 43] explore live execution for accelerated workloads and hardware/software evaluation, but only target specialized stacks.

2.3 Toward a General Live-Simulation Substrate

Modular composition and live execution suggest a general *live-simulation substrate*, which reuses hardware for native functionality and efficient execution, while exposing common interfaces for composing heterogeneous components. The key challenge is that the substrate must coordinate execution and interactions among live and heterogeneous modeled components in a unified manner, while retaining the compatibility and performance advantages of native execution.

Realizing this substrate requires four capabilities.

- *C1: Simulated-time execution control* must coordinate live and modeled execution according to simulated rather than wall-clock time, with a common representation of simulated progress across different execution types.
- *C2: Live-resource control* must prevent unintended host interference and adjust the performance observed from host-backed resources when it differs from the target system.
- *C3: Timed cross-component interaction* must provide common communication interfaces and control the simulated times at which interactions between components become visible.
- *C4: Multi-host coordination* must extend simulated-time progress and timed communication across physical hosts without changing the abstractions exposed to the simulation.

These capabilities are interdependent. Adjusting a live processor’s execution rate to account for target memory latency must also affect its guest-visible time, its progress relative to other components, and the timing of its external interactions. Likewise, a pending message must remain coordinated with the receiver’s progress even when it crosses a physical host boundary. The central challenge is therefore to integrate execution, resource, and interaction control while keeping their common mechanisms independent of component-specific implementations and performance models.

3 Ossim Overview

3.1 OS as a General Live-Simulation Substrate

Our key observation is that the host OS is the natural integration point for all four capabilities identified in §2.3. It provides common control over the execution, physical resources, and communication of heterogeneous live and modeled components, making it possible to provide general simulation abstractions independent of specific components.

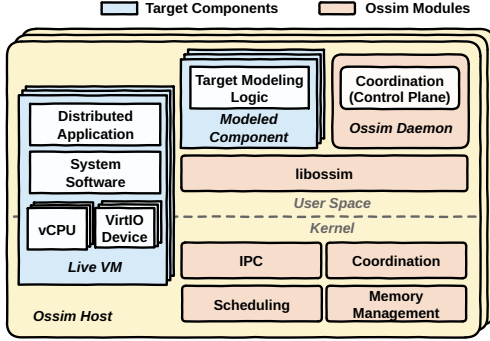


Figure 1. The overall architecture of Ossim.

Simulated-time execution control. Live and modeled components must advance under a common notion of simulated progress. Live VMs execute through a hypervisor under host-OS control, while modeled components typically run as user-space processes scheduled by the kernel. The OS therefore provides a common point for coordinating their execution and time.

Live-resource control. Live components directly use physical resources such as CPUs, memory, caches, and devices. Since the OS manages their allocation, isolation, and access, it is the natural point for controlling host interference and the resource behavior exposed to simulation.

Timed cross-component interaction. Interactions between components must become visible according to simulated time. Although components may use different communication mechanisms, their interactions rely on OS-managed communication and virtual interfaces. The host OS thus provides a common point for controlling interaction timing across heterogeneous components.

Multi-host coordination. At cluster scale, the same execution and interaction semantics must hold across physical hosts. Because these semantics are enforced at each host OS, extending them across machines naturally requires coordination among the participating OS instances.

Together, these observations motivate the host OS as the foundation for a general live-simulation substrate: it provides common control points spanning heterogeneous components, resources, interactions, and physical hosts.

3.2 Ossim Architecture

Figure 1 shows the architecture of Ossim. We implement the OS-native substrate described above in the Linux virtualization stack [41, 53]. Within this stack, hardware virtualization [3, 22] retains native execution and access to exposed processor features, while the host OS controls vCPU scheduling, guest-visible clocks, and the resources backing guest memory. Virtual-device interfaces further connect live VMs with modeled components. Building on these control points,

Table 1. Comparison of simulation methods for cluster-scale full-stack simulation.

Method type	System	Sim. performance	End-host full-stack	Cluster-scale
DES	gem5/Simics	✗	✓	✗
	ns-3/OMNeT++	✓	✗	✓
Modular comp.	SimBricks/	*	✓	✓
	SplitSim			
Live sim.	Phantora	✓	†	✓
	NEX	✓	✗	✗
	Ossim	✓	✓	✓

* Depends on component simulators. † Runs unmodified ML apps without an OS.

Ossim integrates simulation control into the OS while separating generic mechanisms from component-specific implementations.

Ossim comprises four subsystems. *Simulation-oriented scheduling* represents live and modeled components through a common virtual-time abstraction, with domain-based scheduling coordinating their progress (§4). *Live memory hierarchy management* retains native memory accesses and coherence, manages their backing resources, and adjusts virtual-time advancement for target memory-performance differences (§5). *Simulation-aware IPC* provides common interfaces connecting live guests with heterogeneous communication fabric implementations, while controlling when messages become visible to receivers (§6). Finally, *multi-host coordination* extends the above mechanisms across physical machines, allowing a simulation to use resources beyond a single host (§7).

Together, these subsystems realize a general live-simulation substrate that preserves native execution where possible, while bringing execution, resources, and interactions under shared simulation control. Table 1 compares Ossim with existing methods.

4 Simulation-Oriented Scheduling

Ossim coordinates live and modeled execution under shared simulated time (C1) in two steps. First, it represents each coordinated execution thread as a *vtask* and tracks its simulated progress using a common virtual time (*vtime*) (§4.1). Second, it groups *vtasks* into *domains* that bound their *vtime* skew while accommodating temporary inactivity and different synchronization requirements (§4.2).

4.1 Unified Virtual Time Accounting

Vtask abstraction. A *vtask* is a user-space thread whose execution participates in simulated-time coordination. Ossim utilizes this abstraction to decouple *vtime* accounting from the dispatch logic. Each *vtask* exposes a *vtime* to the scheduler, while an execution-specific accounting backend determines how that *vtime* advances. This separation lets the scheduler coordinate heterogeneous execution uniformly.

Ossim supports two accounting backends:

Clock-derived vtime. A *live vtask*, such as a KVM vCPU thread, executes the simulated component directly on hardware and derives vtime from its hardware execution time. A configurable multiplier can scale this mapping when the target component runs at a different speed from the simulation host. For a live vCPU, Ossim implements clock-derived accounting by adapting KVM’s paravirtual clock (pvclock) [54]. Stock KVM advances the guest clock with host wall time even when the vCPU is preempted and reports the preempted periods as steal time; Ossim instead adjusts the TSC (Time Stamp Counter) [22] offset at guest entry so that the clock advances only during guest execution. While the vCPU runs, Ossim extrapolates its vtime from the same clock state. At guest exit, Ossim folds the executed guest time into the vtask’s vtime. Therefore, the guest clock and the scheduler observe the same simulated progress.

The same clock-derived principle also applies to physical devices used as live components. Rather than modeling their internal execution, Ossim measure their physical execution time and translate it into simulated-time progress or delay. For example, our live-DPU case study measures each packet’s physical DPU processing time and schedules its completion after the corresponding vtime delay (§8.4).

Model-driven vtime. A *modeled vtask*, such as a QEMU device-model thread or ns-3 event-loop thread, advances vtime according to its modeling logic. They report accumulated vtime advances through a per-task *run page* shared with the Ossim kernel. On the common path, the kernel consumes pending vtime advances asynchronously to avoid a system call for each report. When a vtask must ensure that an advance has been consumed before continuing, it can instead use a synchronous `ioctl`. Host-side execution itself does not advance a modeled vtask’s vtime. Ossim uses a watchdog that identifies stalled vtasks and emits a warning for faulty components that stop making progress.

4.2 Domain-Based Scheduling

A common vtime makes the progress of heterogeneous vtasks comparable, but coordinating that progress raises several additional challenges. First, runnable vtasks should progress independently when possible to exploit parallelism, while their vtime divergence remains bounded. Second, a vtask that cannot currently make progress should not unnecessarily constrain others. Third, some synchronization policies cannot be enforced through normal scheduler hooks alone. Finally, synchronization requirements may vary across vtasks. Ossim addresses these challenges through domains, which abstract a group of vtasks that must stay synchronized. Figure 2 illustrates these mechanisms using an example simulation with two live VMs and one ns-3 modeled component.

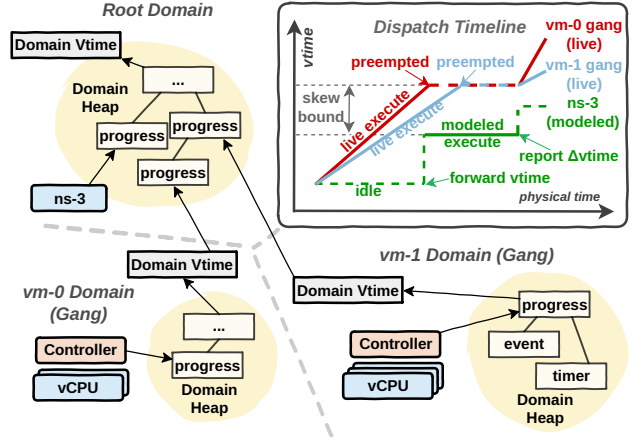


Figure 2. Domain-based scheduling for an example simulation with two live VMs and one modeled ns-3 component. The domain hierarchy and constraints coordinate their progress, while the timeline illustrates skew-bounded dispatch across live and modeled execution.

Skew-bounded dispatch. A domain coordinates its members through constraints on how far they may advance. Ossim represents each constraint as a *constraint node* carrying a simulated-time limit. The domain maintains these nodes in a min-heap, called the *domain heap*. The minimum defines the *domain vtime*.

Each domain D has a *skew bound* $bound_D$, which limits how far its members may progress independently. Analogous to lookahead in conservative PDES [6, 16, 32], it provides a window for independent simulated-time progress. Whereas PDES derives lookahead from lower bounds on future event timestamps, such as communication delays, Ossim applies a configured bound uniformly across a domain. As shown in §8.3, keeping the skew bound below the smallest interaction delay within the domain is important for preserving the timing fidelity of modeled interactions.

The skew bound exposes an accuracy–performance trade-off: a larger bound allows greater independent progress and parallelism, whereas a smaller bound reduces timing error from vtime skew at the cost of more synchronization. The same heap can hold other constraints besides member progress. For example, §6 later uses this mechanism to enforce event visibility.

Handling inactive vtasks. A vtask becomes *inactive* when it cannot progress until a future event, e.g., a halted vCPU or a device awaiting I/O. Ossim then removes its progress node from the heap while preserving its vtime. For timed reactivation, an *event node* records the target vtime; when due, Ossim advances the vtask to that time and restores its progress node. For untimed reactivation, Ossim advances it to the current domain vtime before reinsertion.

Domain controllers. To support more flexible scheduling policies, Ossim allows each domain to attach a *domain controller*, a kernel thread that performs domain-wide coordination beyond ordinary scheduler hooks. The controller can monitor running members, update their scheduling state, and interrupt them when necessary. For example, a live-vCPU domain uses a controller to enforce its skew bound without relying on periodic scheduler ticks that would perturb guest execution. The controller extrapolates the vtime of running vCPUs, updates their progress nodes, and preempts any vCPU that becomes ineligible.

Heterogenous synchronization. Different vtasks within the same domain may require different skew bounds. Applying the tightest requirement to one flat domain would unnecessarily constrain all members, so Ossim supports nested domains. Each child domain enforces its own synchronization policy and participates in its parent as a single member represented by the child’s domain vtime.

Tightly coupled live vtasks may require near-zero vtime skew. Simply using a small skew bound can be inefficient as even a small lead can repeatedly make a member ineligible. Ossim therefore provides a *gang*, a specialized domain whose controller co-schedules its members on separate physical CPUs [13]. Concurrent execution keeps their progress closely aligned, while the gang appears to its parent as one coordinated member.

An SMP live VM is the primary use case for a gang. Its vCPUs share a machine clock and guest memory, so divergent execution progress can expose inconsistent guest-visible timing and abnormal shared-memory behavior. As shown in Figure 2, each live VM forms a gang within the larger simulation domain. The same mechanism can coordinate other tightly coupled components, such as VMs sharing disaggregated memory.

5 Live Memory Hierarchy Management

Ossim keeps memory functional live while modeling its performance (C2). It progressively uses three mechanisms to bridge the gap between host and target memory behavior. First, *cells* isolate each VM within a controlled host-resource substrate (§5.1). Second, *profile-guided vtime scaling* maps memory-stall exposure from host to target timing (§5.2). Finally, *memory-region attribution* assigns stalls to memory regions and scales each by its target characteristics (§5.3).

5.1 Host-Interference Control with Cells

A *cell* is Ossim’s unit of host-resource isolation for a live VM. A cell binds the host resources needed by one live VM. It assigns the VM dedicated CPUs and memory capacity on a host NUMA node, while constraining the shared cache capacity and memory bandwidth available to it. When QEMU attaches a VM, Ossim binds its guest RAM to that node, populates the mapping, and keeps it resident.

Ossim realizes these controls using cpuset-style CPU and NUMA placement [52], memory-capacity admission, and resctrl-based cache and bandwidth allocation. It uses Intel Cache Allocation Technology (CAT) [22] for cache capacity allocation, and Memory Bandwidth Allocation (MBA) [23] constraining memory bandwidth. CPU/NUMA placement controls where the VM executes and where its memory resides, while CAT and MBA constrain the shared resources it consumes. Together, these mechanisms prevent co-located work from unintentionally changing the memory behavior observed by the live VM.

5.2 Profile-Guided Virtual-Time Scaling

Cells remove unintended host interference but not systematic performance differences between host and target memory. For example, host DRAM may be faster than a disaggregated memory. Ossim accounts for this difference by scaling each vCPU’s vtime according to the portion of its execution exposed to memory latency.

Consider an accounting window in which a fraction f of the vCPU’s execution time is exposed memory stall, and the remaining fraction $1 - f$ is unaffected execution. Given a target-to-host memory-latency ratio r , Ossim computes the vtime scale $s = (1 - f) + fr = 1 + f(r - 1)$. Thus, only the execution time exposed to memory latency is scaled; the remaining execution is unchanged.

Ossim estimates f using performance-monitoring unit (PMU) counters. On Intel processors, it tracks `CYCLE_ACTIVITY.STALLS.L3.MISS`, which counts cycles in which execution stalls while a demand load misses the last-level cache [22]. This measures the memory delay exposed to execution while naturally excluding latency hidden by memory-level parallelism. The counter is attached to each vCPU and sampled during guest execution. Ossim smooths the resulting scale with an exponentially weighted moving average and contributes it to the vCPU’s vtime multiplier.

5.3 Memory-Region Attribution Backends

When a guest accesses memory regions with different target-to-host latency ratios, a single aggregate stall fraction is insufficient. We use an emerging configuration with guest-local DRAM and a disaggregated-memory region [9] as a running example. Ossim defines a hardware-independent attribution interface whose backends report per-region stall shares to the common vtime scaler. Ossim currently provides three such backends.

NUMA-source backend. On multi-NUMA Intel hosts, Ossim places guest-local memory on the vCPU’s NUMA node and a disaggregated-memory region on a remote NUMA node. PMU events that distinguish local and remote DRAM accesses [22] allow Ossim to estimate the stall share of each region. Because the remote region already incurs additional

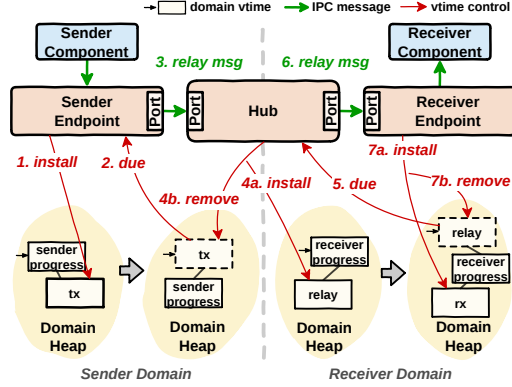


Figure 3. Message path and visibility-time enforcement.

host NUMA latency, Ossim accounts for this cost when mapping its stalls to the target latency. This backend requires suitable NUMA topology and supports one disaggregated-memory class.

Guest-assisted backend. When NUMA placement cannot distinguish memory regions, Ossim performs PEBS attribution inside the guest using a guest agent (§7.1), where sampled virtual addresses identify the accessed region. The guest latency-weights the samples [61] and reports the resulting stall shares to the host, which applies the corresponding target-to-host ratios.

Physical-address backend. On AMD hosts, IBS Op samples memory-access latency together with, when available, the accessed physical address [3]. Ossim maps sampled addresses to registered memory regions and derives their latency-weighted stall shares directly, without requiring NUMA placement or guest-side attribution.

6 Simulation-Aware IPC

Ossim provides simulation-aware IPC so that cross-component communication follows simulated rather than host timing (C3). We now introduce its three fundamental components.

6.1 IPC Abstractions

As shown in Figure 3, Ossim uses *hubs* and *endpoints* to compose components and communication fabrics.

Hubs. A hub models a communication fabric such as Ethernet, routing messages among attached endpoints and capturing the performance effects of traversing the fabric. The hub operates on variants of Ossim messages, which are based on a common abstraction across different fabric protocols.

Endpoints. An endpoint abstracts a component’s communication interface towards the hub, translating between the interface-specific I/O and variants of Ossim messages. Further, it can model the performance effects of traversing an interface such as a NIC. Endpoints attach via the native API different components already expose for integrating with some communication fabric. For an Ethernet interface, for

example, a live VM exposes a QEMU vhost-user device as the API, whereas a modeled component such as ns-3 exposes file-descriptor net devices.

Ports. Hubs and endpoints have ports, which connect pairwise to exchange the Ossim messages. Ports provide only this relay interface and perform neither routing nor performance modeling and are therefore oblivious to the Ossim message variant being used.

6.2 Enforcing Visibility-Time Delivery

To synchronize communication in simulated time, Ossim attaches a *visibility vtime* to each message, which indicates when the next node in the communication path may observe it. Ossim enforces these times using the domain vtime heaps from §4.

Each endpoint is associated with its component’s domain. When an endpoint-local operation, such as NIC transmission or reception, incurs modeled delay, the endpoint buffers the message and inserts an event node at its visibility vtime. In Figure 3, the sender endpoint first installs the *tx* node (1). When the node becomes due (2), the endpoint relays the message to the hub (3).

Each relay establishes the next vtime constraint in its destination domain before the constraint in the source domain to ensure that an in-flight message always has a vtime constraint in some domain, preventing simulation progress from overtaking the visibility vtime. After routing the message and applying the modeled fabric delay, the hub therefore installs an event node at the resulting visibility vtime in the receiver domain *before* releasing the sender-side constraint (4a–4b).

6.3 Modeling the Communication Fabric

Communication fabrics vary widely in modeling complexity: some require only parameterized link behavior, while others require programmable forwarding or a detailed external simulator. The hub abstraction lets Ossim support these choices without changing the endpoint and port interfaces. Ossim supports three types of fabric implementation: built-in kernel models, programmable eBPF models, and external modeled components.

Built-in models. Built-in hubs implement forwarding and timing entirely in the kernel and expose configurable performance parameters. For Ethernet, Ossim provides a point-to-point link and a switch. They suit common fabrics whose behavior is fixed while performance parameters vary.

Programmable models. For customized fabrics, Ossim uses a per-message eBPF hook for hub forwarding and timing while keeping the message path in the kernel. The hook selects the destination port and updates visibility vtime, with eBPF maps storing topology, routing, and contention state. Ossim provides `eth_ebpf` for this purpose.

Modeled fabrics. More complex fabrics can run as external modeled components with their own state and event loops.

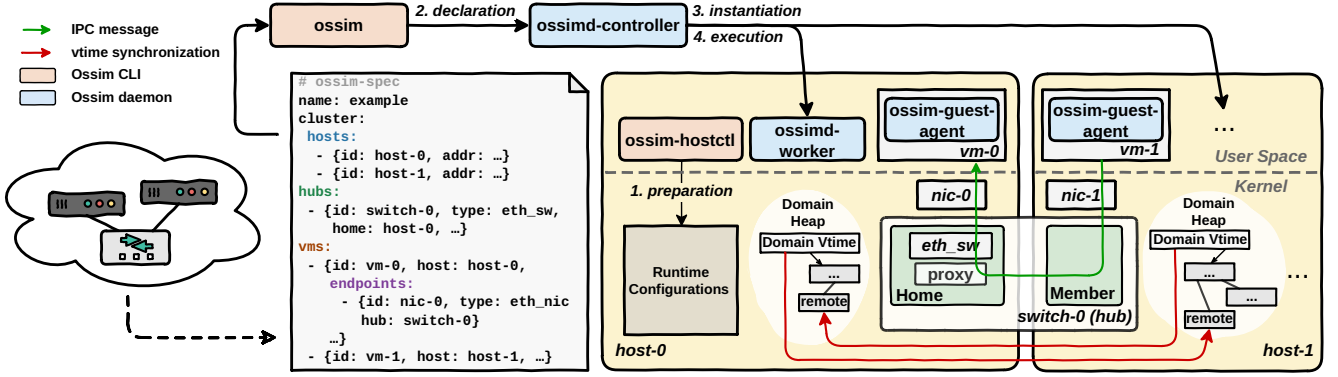


Figure 4. Running one simulation on a Ossim cluster. The control plane carries the simulation from preparation to execution, after which the worker kernels exchange frontier updates and IPC messages directly.

They connect through the same endpoint–hub interface, while Ossim enforces visibility-time delivery at the boundary. For example, an ns-3 topology can run as a modeled component connected to live-VM endpoints through point-to-point link hubs, while retaining ns-3’s internal scheduler and network models.

7 Multi-Host Coordination

Ossim scales beyond one physical machine by coordinating hosts into a Ossim cluster (C4). Ossim separates multi-host coordination into two parts: a kernel *data plane* preserves simulation semantics across host boundaries during execution (§7.1), while a user-space *control plane* configures the cluster and manages the lifecycle of a simulation (§7.2).

7.1 The Data Plane

The data plane extends Ossim’s local semantics across host boundaries. Though memory isolation remains host-local, the scheduling and IPC guarantees of Ossim must be preserved with additional mechanisms to span across participating hosts. Ossim preserves these guarantees by having host kernels exchange vtime progress and timed IPC messages directly through inter-host channels.

Cross-host vtime synchronization. Cross-host vtime synchronization preserves bounded-skew execution across hosts. Each host publishes its root domain’s vtime to its peers, which represent the received value with a *remote constraint node* in their local domain heap. This node constrains domain progress under the same bounded-skew dispatch rule in §4.2. A host sends an update when its root-domain vtime advances sufficiently. To deal with the very uncommon case of lost updates, a host periodically retransmits its latest vtime.

Cross-host IPC. Cross-host IPC extends a logical hub across hosts without changing its interface or timing semantics. Endpoints attach to the same hub regardless of where their peers reside, while the hub engine continues to determine routing and message visibility times as in §6. Thus, distribution remains transparent to both endpoints and hub engines.

A cross-host hub represents remote endpoints through *proxy endpoints*. Ossim realizes one logical hub with a *home* instance and *member* instances on remote hosts. At the home instance, each remote endpoint is represented by a proxy endpoint, allowing the hub engine to process it exactly like a local one. Ossim then transports the message and its visibility metadata to the corresponding member using reliable kernel TCP. This preserves modeled loss semantics while allowing the same distribution mechanism to compose with different hub engines without distributed variants of their modeling logic.

7.2 The Control Plane

The control plane turns a cluster-wise simulation description into a running experiment. Each host runs a worker daemon (*ossimd-worker*) that owns its local simulation resources, and one controller daemon (*ossimd-controller*) coordinates operations across workers. Every Ossim host runs as a worker that owns its local simulation resources, while a controller coordinates operations across them. A client accesses the Ossim cluster through the controller. As Figure 4 shows, a simulation proceeds through preparation, declaration, instantiation, and execution.

Preparation. Preparation establishes a known measurement state on each host. *ossim-hostctl* administers each host before it participates in a simulation, configuring and verifying measurement-relevant settings such as CPU power management, idle states, and reserved CPUs. For machine-dependent parameters, *ossim-hostctl* derives appropriate values using a bundled host profiler.

Declaration. Declaration specifies both the target simulation and its mapping onto the physical cluster. A YAML specification submitted through *ossim* first describes the logical simulation: its components, their configurations, IPC hubs, and connections. It maps these objects onto physical resources by naming participating workers and specifying component placement and resource assignments. *ossimd-controller* resolves the specification into a cluster-wide

plan and validates its references, placement, and resource-sharing constraints before sending to the workers.

Instantiation. Instantiation creates the resources required by the validated simulation specification. The controller sequences cluster-wide operations, while each worker creates and owns the component processes, kernel objects, and other backing resources that are assigned. A prepare/commit barrier completes instantiation only after every worker reports that its local resources are ready.

Execution. Execution runs the instantiated system and controls workloads inside its components. To avoid paying synchronization overhead during guest boot and experiment setup, Ossim can initially run components without vtime synchronization and switch to synchronized execution before entering the target region of interest. The optional in-guest ossim-guest-agent communicates over vsock and supports features such as command execution and result collection. It also carries the guest-assisted memory-attribution reports described in §5. For live VMs without ossim-guest-agent, Ossim provides a console interface as a fallback for interactive guest control.

8 Evaluation

We evaluate whether Ossim combines full-stack fidelity with practical simulation performance. We first measure end-to-end accuracy and speed (§8.2), then isolate the mechanisms and their accuracy–performance tradeoffs (§8.3). Finally, two case studies examine composition with different network models and live hardware (§8.4).

8.1 Implementation and Experimental Setup

Implementation. Ossim adds 42 K lines to Linux, 3.0 K to QEMU, 2.3 K to ns-3, and 13.7 K in libossim for control-plane and operator support, plus roughly 15 K lines of build, operator, and workload tooling. Coding agents assisted implementation and debugging.

Testbed and compared setups. Experiments use three identical hosts, each with dual Intel Xeon Gold 6326 processors (64 CPUs across two NUMA nodes) and a Mellanox ConnectX-5 NIC, connected through a switch with 10 Gb/s links. We compare Ossim with *physical*, the physical ground truth; *stock KVM*, unmodified KVM/QEMU with Linux-bridge networking and netem for latency or bandwidth shaping when needed; and *gem5 composition*, SimBricks [32] with gem5 end hosts. The hardware-backed setups use the same hosts, Ubuntu 24.04 LTS, and Linux 6.18; stock KVM uses QEMU 8.2. Target machines default to 4 CPUs and 16 GB of DRAM.

A single pre-experiment run of Ossim’s host profiler calibrates machine-dependent parameters (§7.2). All Ossim experiments reuse this profile, which is independent of the workloads and their target results.

Table 2. End-to-end fidelity and simulation performance. N is the number of server instances.

Workload	N	Metric	Physical	Stock KVM (err)	Ossim (err)	Wall / Vtime (s)
memcached	2	p50 (μ s)	66	127 (93.4%)	66 (0.0%)	155 / 60
Hotel (K8s)	2	p50 (μ s)	76	107 (40.2%)	81 (6.1%)	163 / 60
Hotel (dist. K8s)	3	p50 (μ s)	61	95 (54.9%)	58 (5.4%)	171 / 60
TPC-C (MySQL)	2	p50 (ms)	7.74	13.01 (68.1%)	7.63 (1.5%)	623 / 60
		tpmC	38,706	24,118 (37.7%)	39,466 (2.0%)	
Redis						
primary/backup	3	p50 (μ s)	84	106 (25.3%)	87 (3.2%)	167 / 60
TPC-DS 99 (Spark)	3	runtime (s)	5.09	20.07 (294.4%)	4.63 (9.0%)	162 / 6.72

8.2 End-to-End Fidelity and Performance

Ossim runs unmodified distributed stacks with end-to-end metrics within 9% of physical measurements. Table 2 covers memcached, Kubernetes-based hotel reservation on one or multiple hosts, MySQL TPC-C, Redis primary/backup, and a three-instance Spark TPC-DS workload. Across seven reported metrics, its median error is 3.2%, whereas stock KVM deviates by 25–294%. Software compatibility alone therefore does not preserve target performance; simulation must also control execution, resources, and communication.

This fidelity remains practical to obtain: Ossim requires 2.6–24 host seconds per simulated second, whereas no corresponding gem5 end-to-end experiment completes within the 24-hour timeout.

8.3 Accuracy and Performance Microbenchmarks

To explain these end-to-end results, we isolate scheduling, memory, and communication control (Figure 5), then examine simulation cost and skew-bound sensitivity (Figure 6). Accuracy is measured against matching physical hardware when available and configured models otherwise. Because the gem5 CPU models are not calibrated to our processors, we use them only for simulation-performance comparisons.

Scheduling (§4). Ossim’s simulation-aware scheduling keeps guest execution and wake-up timing close to physical under host CPU contention. We run sysbench CPU, threads, and mutex tests, a 1 ms periodic timer, and a two-thread futex ping-pong while background work consumes 0, 10, or 20% of each vCPU’s physical core.

Figure 5(a)–(c) show the effect on guest execution. Both stock KVM and Ossim stay close to physical without background load. As load increases, stock KVM suffers lower throughput or longer runtime. Ossim instead stays near its isolated result because guest vtime advances with accounted execution rather than host wall time.

For wake-up timing, Figure 5(d)–(e) shows the contrast appears even without background load. Maximum timer wake-up latency is 10.4 $\times$ physical under stock KVM but only 1.1 $\times$ under Ossim. Background load further raises the stock-KVM timer latency to 56 $\times$ physical and increases futex p99 latency, whereas Ossim’s coordinated dispatch keeps both probes close to physical.

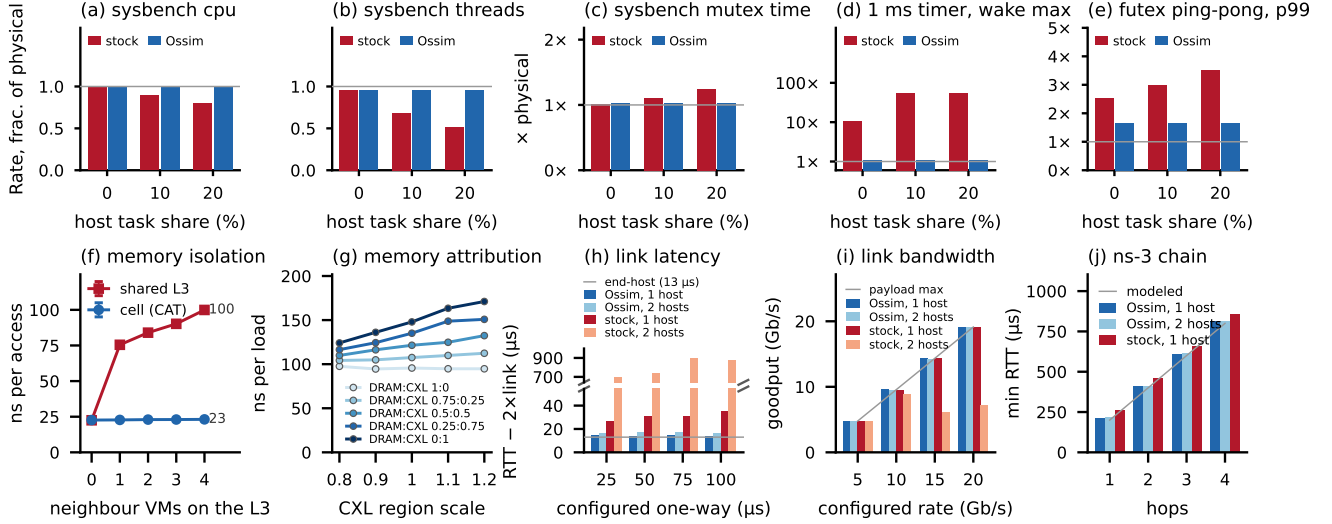


Figure 5. Mechanism-level accuracy. (a)–(e) evaluate scheduling under different host CPU background load; (f)–(g) evaluate memory interference isolation and per-region performance scaling; (h)–(j) evaluate modeled communication on one host and across hosts. Thin gray lines indicate the optimal case in (a)–(e), (i), and (j), and the measured end-host delay in (h).

Memory management (§5). Beyond execution timing, Ossim controls both memory interference and region-specific performance. We first test isolation with a dependent-load pointer chase in one VM, while up to four neighboring VMs stream through the last-level cache on other cores of the same socket (Figure 5(f)). Without isolation, latency rises from 22.5 to 100 ns; by enforcing cells, it remains 22.7–23.1 ns, within 3% of the isolated case.

Whereas isolation stabilizes the live memory substrate, per-region scaling adjusts its modeled performance (Figure 5(g)). A memory-intensive chase mixes local DRAM and CXL accesses while only the CXL region’s scale varies from 0.8 to 1.2. All DRAM workloads remain stable, all CXL workloads track the scale, and mixed workloads follow targets weighted by their CXL access shares. This validates Ossim’s region-specific stall attribution and timing adjustment.

IPC and multi-host coordination (§6, §7). Beyond controlling each live VM, Ossim must preserve modeled communication between components, including across hosts. For Figure 5(h)–(i), stock KVM uses a Linux bridge with netem and runs without background load.

Figure 5(h) first evaluates modeled latency. We vary one-way link latency from 25 to 100 μ s and report RTT - 2 $\times$ link latency, which should remain near the measured 13 μ s end-host contribution. Ossim leaves only 14–15 μ s on one host and 16–17 μ s across hosts, whereas stock KVM leaves 27–35 μ s and 693–892 μ s, respectively. This gap arises since netem adds modeled delay atop the physical path, whereas Ossim controls message visibility in simulated time.

This decoupling from the physical path also lets Ossim reproduce modeled bandwidth beyond the physical 10 Gb/s

link (Figure 5(i)). Both setups match the configured bandwidth on one host, but across hosts stock KVM is capped by the physical link. In contrast, for modeled 15 and 20 Gb/s links, Ossim delivers 14.20 and 19.10 Gb/s, within 1.1% of the respective payload maxima of 14.35 and 19.13 Gb/s.

Finally, Figure 5(j) shows that the same timing control extends to an external ns-3 network model. For a chain adding 100 μ s one-way latency per hop, both systems reproduce the expected 200 μ s RTT increase per hop. However, Ossim remains only 12–14 μ s above the model, versus 57–60 μ s for stock KVM with ns-3’s TAP integration. Distributing Ossim’s VMs across two hosts, with ns-3 on one of them, adds only about 1 μ s while preserving the same slope. Our stock-KVM baseline has no corresponding two-host configuration because its ns-3 TAP integration connects the live guests and ns-3 only within one host.

Simulation performance. Whereas the preceding experiments evaluate accuracy, Figure 6(a) examines the simulation-performance benefit of live end-host execution over gem5 composition. For compute-intensive sysbench, increasing SMP width from one to three CPUs raises gem5’s cost with both TimingSimple and O3 models. In contrast, Ossim’s cost remains nearly flat because additional vCPUs run directly on additional physical cores in parallel.

We therefore give every simulated host only one CPU in the distributed gem5 experiments, which favors the gem5 composition. Even with this configuration, Ossim requires 5.5–25 wall-clock seconds per simulated second, compared with 665–1426 for gem5 composition with SimpleTiming CPU and 3381–9329 for O3. Across the corresponding workloads, Ossim is 57–121 $\times$ faster than TimingSimple and 360–1032 $\times$ faster than O3. This shows that live execution removes

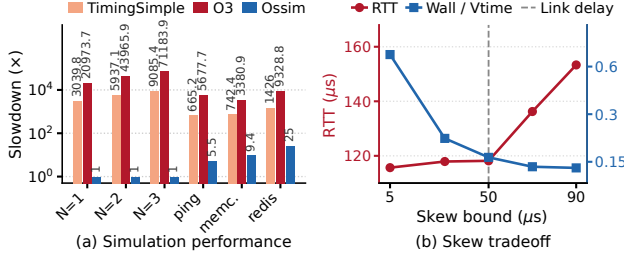


Figure 6. (a) Simulation performance against gem5 composition. (b) RTT and simulation cost (wall-clock seconds per simulated second) versus skew bound; the dashed line marks the modeled one-way latency.

the end-host simulation cost that otherwise dominates a full-stack modular composition under typical workloads.

Skew-bound tradeoff. Live execution avoids detailed CPU simulation, but its coordination cost still depends on how tightly components synchronize. Figure 6(b) examines the accuracy–performance tradeoff by sweeping the bound from 5 to 90 μ s for two VMs in separate gangs. The VMs exchanging pings over a link with 50 μ s one-way latency. Increasing the bound from 5 to 50 μ s keeps RTT near 118 μ s while reducing cost from 0.715 to 0.160 host seconds per simulated second (4.5 $\times$). Beyond the modeled one-way delay, however, RTT rises by 18 and then 35 μ s for little reduction in cost. These results suggest choosing a bound no larger than the smallest modeled one-way delay relevant to the experiment.

8.4 Case Studies

Having examined fidelity and performance, we demonstrate the flexibility and generality of Ossim with two case studies. We first compose the same live VMs with different underlying network implementations, and then add a physical DPU as a live component.

Composing a fat-tree network. The first case study changes the network model without changing VM interfaces. We connect sixteen one-vCPU VMs to a $k=4$ fat-tree with 10 μ s latency per hop, using either an eBPF hub or external ns-3 component (§6.3). The eBPF hub implements topology, routing, and timing in a programmable hub, whereas ns-3 provides the network as an external modeled component which allows adding additional background traffic with modeled hosts. As Table 3(a) shows, both network implementations reproduce the expected 40 μ s RTT increase between topology tiers.

Composing a physical DPU. Whereas the fat-tree case varies a modeled fabric, the second case extends live execution to a physical BlueField-2 DPU for packet processing offloading. Ossim provides an in-kernel backend that connects the physical DPU to a VM’s NIC endpoint through two dedicated virtual functions (VFs). Packets are submitted through one VF and the DPU’s rewritten packets are received

Table 3. Case studies. (a) Round-trip time between VM pairs in the $k=4$ fat-tree. (b) Round-trip time of NAT through the gateway VM’s CPU and DPU paths.

(a) Fat-tree RTT (μ s)			(b) NAT RTT (μ s)			
VM pair	eBPF	ns-3	Workers	CPU	DPU-SW	DPU-ASIC
Same edge	59.7	57.8	0	70.04	72.71	54.89
Same pod	99.4	98.1	4	149.16	71.43	53.95
Cross pod	139.8	137.5	8	163.12	72.02	54.29

through the other. For traffic that comes from the opposite direction, the VF roles reverse. The backend measures each packet’s DPU processing time and schedules completion according to it in vtime.

Using this integration, we compare NAT in a gateway VM’s Linux stack, DPU software, and the DPU ASIC. Three live VMs—client, gateway, and server—connect through an Ethernet switch hub; the DPU paths bypass the gateway’s network stack. To compare sensitivity to gateway load, we place 1,024 nonmatching NAT rules before the matching rule and stress the gateway with workers that run inside it, measuring 100 UDP echoes per configuration after warmup. CPU-path RTT rises from 70.04 to 163.12 μ s with increasing load, while DPU software and ASIC paths remain near 72 and 54 μ s (Table 3(b)).

9 Related Work and Discussion

Simulation acceleration. Apart from live simulation, prior work accelerates simulation through system and runtime optimizations [6, 11, 17, 33, 35, 37], learned performance models [14, 31, 46, 59, 62], and specialized hardware such as GPUs and FPGAs [7, 20, 26, 28, 34, 42].

Simulation for distributed stacks. Other work targets specific cluster workloads, particularly GPU training [19, 43, 56], by simulating both distributed training strategies and the underlying hardware behavior. In contrast, Ossim focuses on unmodified full-stack distributed software and its interaction with underlying hardware at cluster scale.

Resource isolation for guests. Partitioning hypervisors [8, 27, 38, 44] statically bundle fixed host resources for each guest and isolate them through strict non-sharing of cache and memory hardware. In contrast, Ossim dynamically allocates host resources as simulated guests come and go to preserve simulation accuracy under controlled sharing.

Discussion. Oversubscribing a cell would require *temporal* sharing of the live memory hierarchy. Even if the scheduler time-slices vCPUs, an incoming live VM can inherit cache and prefetcher state from its predecessor. Preserving fidelity would require reconditioning that state. Temporal cache reconditioning remains outside this work, and Ossim does not support safe cell oversubscription. Beyond KVM-based VMs, integrating other virtualization technologies [10, 12, 39, 47, 58, 60, 63] could extend Ossim to additional live hardware such as GPUs.

10 Conclusion

Ossim takes an OS-level approach to cluster-scale full-stack simulation of unmodified distributed software stacks by integrating simulation-oriented mechanisms into the Linux virtualization stack. Ossim keeps production software live while OS mechanisms control simulated time, memory, communication, and their distribution across hosts. We see Ossim as an initial step toward *simulation-native OS support*, a broader direction beyond cluster-scale full-stack simulation.

References

- [1] 2025. Ultra Ethernet Specification v1.0.1. <https://ultraethernet.org/uec-1-0-spec> Accessed: 2026-05-18.
- [2] 2026. Ultra Accelerator Link Specifications. <https://ualinkconsortium.org/specification/>. Accessed: 2026-05-18.
- [3] Advanced Micro Devices, Inc. 2026. *AMD64 Architecture Programmer's Manual, Volume 2: System Programming*. Publication 24593, Rev. 3.42; accessed 2026-08-24.
- [4] Apache Software Foundation. 2026. Apache Hadoop: Open-Source Framework for Distributed Storage and Processing. <https://hadoop.apache.org/> Accessed: 2026-05-18.
- [5] Apache Software Foundation. 2026. Apache Spark: Unified Analytics Engine for Large-Scale Data Processing. <https://spark.apache.org/> Accessed: 2026-05-18.
- [6] Songyuan Bai, Hao Zheng, Chen Tian, Xiaoliang Wang, Chang Liu, Xin Jin, Fu Xiao, Qiao Xiang, Wanchun Dou, and Guihai Chen. 2024. Unison: A Parallel-Efficient and User-Transparent Network Simulation Kernel. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*.
- [7] Ben Romdhanne Bilel and Nikaein Navid. 2012. Cunesim: A gpu based simulation testbed for large scale mobile networks. In *2012 International Conference on Communications and Information Technology (ICCIT)*. IEEE, 374–378.
- [8] T. L. Borden, J. P. Hennessy, and J. W. Rymarczyk. 1989. Multiple Operating Systems on One Processor Complex. *IBM Systems Journal* 28, 1 (1989), 104–123. doi:10.1147/sj.281.0104
- [9] CXL Consortium. 2025. Compute Express Link (CXL) Specification Revision 4.0. <https://www.computeexpresslink.org/download-the-specification>. Accessed: 2026-05-19.
- [10] Niklas Eiling, Jonas Baude, Stefan Lankes, and Antonello Monti. 2022. Cricket: A virtualization layer for distributed execution of CUDA applications with checkpoint/restart support. *Concurrency and Computation: Practice and Experience* 34, 14 (2022), e6474. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.6474> doi:10.1002/cpe.6474
- [11] Fares Elsabbagh, Shabnam Sheikhha, Victor A Ying, Quan M Nguyen, Joel S Emer, and Daniel Sanchez. 2023. Accelerating rtl simulation with hardware-software co-design. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 153–166.
- [12] Ruwen Fan, Tingxu Ren, Minhui Xie, Shiwei Gao, Jiwu Shu, and Youyou Lu. 2025. GPREENPT: GPU Preemptive Scheduling Made General and Efficient. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 263–272.
- [13] Dror G Feitelson and Larry Rudolph. 1992. Gang scheduling performance benefits for fine-grain synchronization. *Journal of Parallel and distributed Computing* 16, 4 (1992), 306–318.
- [14] Miquel Ferriol-Galmés, Jordi Paillisse, José Suárez-Varela, Krzysztof Rusek, Shihan Xiao, Xiang Shi, Xiangli Cheng, Pere Barlet-Ros, and Albert Cabellos-Aparicio. 2023. RouteNet-Fermi: Network modeling with graph neural networks. *IEEE/ACM transactions on networking* 31, 6 (2023), 3080–3095.
- [15] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, Vivek Bhanu, Adrian Caulfield, Eric Chung, et al. 2018. Azure accelerated networking: {SmartNICs} in the public cloud. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. 51–66.
- [16] Richard M Fujimoto. 1990. Parallel discrete event simulation. *Commun. ACM* 33, 10 (1990), 30–53.
- [17] Kaihui Gao, Li Chen, Dan Li, Vincent Liu, Xizheng Wang, Ran Zhang, and Lu Lu. 2023. Dons: Fast and affordable discrete event network simulation with automatic parallelization. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 167–181.
- [18] gem5 Project. 2026. The gem5 Simulator Project. <https://www.gem5.org/>. Accessed: 2026-05-19.
- [19] Fei Gui, Kaihui Gao, Li Chen, Dan Li, Vincent Liu, Ran Zhang, Hongbing Yang, and Dian Xiong. 2025. Accelerating Design Space Exploration for {LLM} Training Systems with Multi-experiment Parallel Simulation. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 473–488.
- [20] Zizheng Guo, Yanqing Zhang, Runsheng Wang, Yibo Lin, and Haoxing Ren. 2025. GEM: GPU-accelerated emulator-inspired RTL simulation. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.
- [21] Diwaker Gupta and Kashi V. Vishwanath. 2008. DieCast: Testing Distributed Systems with an Accurate Scale Model. In *5th USENIX Symposium on Networked Systems Design and Implementation (NSDI '08)*. USENIX Association, San Francisco, CA.
- [22] Intel Corporation. 2026. *Intel(R) 64 and IA-32 Architectures Software Developer's Manual*. Accessed: 2026-05-19.
- [23] Intel Corporation. 2026. Introduction to Memory Bandwidth Allocation. <https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-memory-bandwidth-allocation.html>. Accessed: 2026-05-20.
- [24] Intel Simics. 2026. Simics Full-System Simulator. <https://www.intel.com/content/www/us/en/developer/articles/tool/simics-simulator.html>. Accessed: 2026-05-19.
- [25] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [26] Sagar Karandikar, Howard Mao, Donggyu Kim, David Biancolin, Alon Amid, Dayeol Lee, Nathan Pemberton, Emmanuel Amaro, Colin Schmidt, Aditya Chopra, et al. 2018. FireSim: FPGA-accelerated cycle-exact scale-out system simulation in the public cloud. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 29–42.
- [27] Eric Keller, Jakub Szefer, Jennifer Rexford, and Ruby B. Lee. 2010. NoHype: Virtualized Cloud Infrastructure without the Virtualization. In *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA '10)*. 350–361. doi:10.1145/1815961.1816010
- [28] Sajj Khashab, Hariharan Sezhian, Rani Abboud, Alex Normatov, Stefan Kaestle, Eliav Bar-Ilan, Mohammad Nassar, Omer Shabtai, Wei Bai, Matty Kadosh, et al. 2025. NSX: Large-Scale Network Simulation on an AI Server. In *Proceedings of the 2nd Workshop on Networks for AI Computing*. 19–25.
- [29] Jereme Lamps, Vladimir Adam, David M. Nicol, and Matthew Caesar. 2015. Conjoining Emulation and Network Simulators on Linux Multiprocessors. In *Proceedings of the 3rd ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (London, United Kingdom) (SIGSIM PADS '15)*. Association for Computing Machinery, New York, NY, USA, 113–124. doi:10.1145/2769458.2769481
- [30] Jereme Lamps, David M. Nicol, and Matthew Caesar. 2014. Time-Keeper: a lightweight virtual time system for linux. In *Proceedings*

- of the 2nd ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (Denver, Colorado, USA) (SIGSIM PADS '14). Association for Computing Machinery, New York, NY, USA, 179–186. doi:10.1145/2601381.2601395
- [31] Chenning Li, Arash Nasr-Esfahany, Kevin Zhao, Kimia Noorbakhsh, Prateesh Goyal, Mohammad Alizadeh, and Thomas E Anderson. 2024. m3: Accurate flow-level performance estimation using machine learning. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 813–827.
- [32] Hejing Li, Jialin Li, and Antoine Kaufmann. 2022. SimBricks: end-to-end network system evaluation with modular simulation. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 380–396.
- [33] Hejing Li, Marvin Meiers, Jialin Li, and Antoine Kaufmann. 2025. SplitSim: Towards Practical Large-Scale Full-System Simulation for Systems Research. *Proceedings of the ACM on Networking* 3, CoNEXT4 (2025), 1–19. doi:10.1145/3768999
- [34] Qinyong Li, Zhiwei Zhao, Geyong Min, Zi Wang, and Luwei Fu. 2026. GeDES: GPU-Driven Discrete Event Network Simulator. In *Proceedings of the 21st European Conference on Computer Systems*. 468–483.
- [35] Tiantian Lin, Cheng Qiu, Xiaohang Wang, Ling Wang, Zhulin Zheng, Yingtao Jiang, Amit Kumar Singh, Jieming Yin, Sihai Qiu, Xiaodong Li, et al. 2025. LEGOSim: A Unified Parallel Simulation Framework for Multi-chiplet Heterogeneous Integration. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 1347–1362.
- [36] Jiacheng Ma, Jonas Kaufmann, Emilien Guandalino, Rishabh Iyer, Bourgeat Thomas, and George Candea. 2025. Fast End-to-End Performance Simulation of Accelerated Hardware–Software Stacks. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*.
- [37] Tie Ma, Long Luo, Hongfang Yu, Xi Chen, Jingzhao Xie, Chongxi Ma, Yunhan Xie, Gang Sun, Tianxi Wei, Li Chen, et al. 2024. Klonet: an {Easy-to-Use} and scalable platform for computer networks education. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 2025–2046.
- [38] José Martins, Adriano Tavares, Marco Solieri, Marko Bertogna, and Sandro Pinto. 2020. Bao: A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems. In *Workshop on Next Generation Real-Time Embedded Systems (NG-RES), co-located with HiPEAC*. 3:1–3:14. doi:10.4230/OASlcs.NG-RES.2020.3
- [39] NVIDIA Corporation. 2026. NVIDIA Virtual GPU (vGPU) Software. <https://docs.nvidia.com/vgpu/>. Accessed: 2026-05-19.
- [40] OpenInfra Foundation. 2026. The Most Widely Deployed Open Source Cloud Software in the World. <https://www.openstack.org> Accessed: 2026-05-19.
- [41] QEMU Project. 2026. QEMU: The Fast Processor Emulator. <https://www.qemu.org/>. Accessed: 2026-05-18.
- [42] Yicheng Qian, Ran Shu, Rui Ma, Yang Wang, Derek Chiou, Nadeen Gebara, Luca Piccolboni, Miriam Leeser, and Yongqiang Xiong. 2025. Miniature: Fast AI Supercomputer Networks Simulation on FPGAs. In *Proceedings of the 9th Asia-Pacific Workshop on Networking*. 114–120.
- [43] Jianxing Qin, Jingrong Chen, Xinhao Kong, Yongji Wu, Tianjun Yuan, Liang Luo, Zhaodong Wang, Ying Zhang, Tingjun Chen, Alvin R Lebeck, et al. 2026. Phantora: Maximizing Code Reuse in Simulation-based Machine Learning System Performance Estimation. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 1809–1825.
- [44] Ralf Ramsauer, Jan Kiszka, Daniel Lohmann, and Wolfgang Mauerer. 2017. Look Mum, no VM Exits! (Almost). In *Proceedings of the 13th Workshop on Operating Systems Platforms for Embedded Real-Time Applications (OSPERT)*. <https://arxiv.org/abs/1705.06932>.
- [45] George F. Riley and Thomas R. Henderson. 2010. The ns-3 Network Simulator. In *Modeling and Tools for Network Simulation*. Springer, 15–34. doi:10.1007/978-3-642-12331-3_2
- [46] Krzysztof Rusek, José Suárez-Varela, Paul Almasan, Pere Barlet-Ros, and Albert Cabellos-Aparicio. 2020. RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN. *IEEE Journal on Selected Areas in Communications* 38, 10 (2020), 2260–2270.
- [47] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. {XSched}: Preemptive Scheduling for Diverse {XPU}s. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 671–692.
- [48] Jovan Stojkovic, Abraham Farrell, Zhangxiaowen Gong, Christopher J Hughes, and Josep Torrellas. 2026. AccelFlow: Orchestrating an On-Package Ensemble of Fine-Grained Accelerators for Microservices. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–17.
- [49] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, et al. 2023. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 105–121.
- [50] The libvirt Project. 2026. libvirt virtualization API. <https://libvirt.org/> Accessed: 2026-05-19.
- [51] The Linux Foundation. 2026. Kubernetes: Production-Grade Container Orchestration. <https://kubernetes.io/> Accessed: 2026-05-19.
- [52] The Linux Kernel Foundation. 2026. Cpusets. <https://docs.kernel.org/admin-guide/cgroup-v1/cpusets.html>. Accessed: 2026-05-20.
- [53] The Linux Kernel Foundation. 2026. KVM: Kernel-based Virtual Machine. <https://www.linux-kvm.org/>. Accessed: 2026-05-18.
- [54] The Linux Kernel Foundation. 2026. KVM PVclock. <https://www.linux-kvm.org/page/KVMClock>. Accessed: 2026-05-20.
- [55] András Varga and Rudolf Hornig. 2008. An Overview of the OM-NeT++ Simulation Environment. In *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTOOLS '08)*. ICST, Marseille, France, 1–10. doi:10.4108/ICST.SIMUTOOLS2008.3027
- [56] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, Linkang Zheng, Sen Zhang, Yikai Zhu, et al. 2025. {SimAI}: unifying architecture design and performance tuning for {Large-Scale} large language model training with scalability and precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 541–558.
- [57] Elias Weingärtner, Florian Schmidt, Hendrik vom Lehn, Tobias Heer, and Klaus Wehrle. 2011. SliceTime: A Platform for Scalable and Accurate Network Emulation. In *8th USENIX Symposium on Networked Systems Design and Implementation (NSDI '11)*. USENIX Association, Boston, MA.
- [58] Yechen Xu, Yifei Wang, Nathanael Ren, Yiran Chen, and Danyang Zhuo. 2026. Nixie: Efficient, Transparent Temporal Multiplexing for Consumer GPUs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 2085–2101.
- [59] Qingqing Yang, Xi Peng, Li Chen, Libin Liu, Jingze Zhang, Hong Xu, Baochun Li, and Gong Zhang. 2022. DeepQueueNet: Towards scalable and generalized network performance estimation with packet-level visibility. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 441–457.
- [60] Yanan Yang, Zhengxiong Jiang, Meiqi Zhu, Hongqiang Xu, Yujun Wang, Liang Li, Jiansong zhang, and Jie Wu. 2026. gShare: Efficient GPU Sharing with Aggressive Scheduling in Multi-tenant FaaS platform. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (USA) (ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 838–859. doi:10.1145/3779212.3790168
- [61] Quan Yi, Xu Liu, and Shengtian Yao. 2020. On the Precision of Precise Event Based Sampling. In *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '20)*. 98–105. doi:10.1145/3409963.3410974
- [62] Qizhen Zhang, Kelvin KW Ng, Charles Kazer, Shen Yan, João Sedoc, and Vincent Liu. 2021. MimicNet: Fast performance estimates for data

- center networks with machine learning. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 287–304.
- [63] Shulai Zhang, Ao Xu, Quan Chen, Han Zhao, Weihao Cui, Zhen Wang, Yan Li, Limin Xiao, and Minyi Guo. 2025. Efficient Performance-Aware GPU Sharing with Compatibility and Isolation through Kernel Space Interception. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 1003–1019.